

2. Лабораторная работа №2. Изучение перегрузки стандартных операций в языке Си++

2.1. Цель и задачи работы, требования к результатам ее выполнения

Цель работы состоит в овладении навыками разработки программ на языке Си++, использующих перегрузку стандартных операций. Для достижения цели необходимо выполнить следующие задачи:

- изучить необходимые учебные материалы, посвященные перегрузке стандартных операций в языке Си++ ;
- разработать программу на языке Си++ для решения заданного варианта задания;
- отладить программы;
- выполнить решение контрольного примера с помощью программы и ручной расчет контрольного примера;
- подготовить отчет по лабораторной работе.

2.2. Краткая характеристика объекта изучения

Перегрузка операций в языке Си++ это возможность распространения действия стандартных операций на операнды, для которых эти операции первоначально не предназначались. Это возможно, если хотя бы один из операндов является объектом класса, для этого создается специальная, так называемая, оператор- функция, которая может быть как членом класса, так и функцией, не принадлежащей классу.

Формат определения оператор- функции имеет вид:

<тип_возвращаемого_значения>

operator <знак_операции>

(спецификация_параметров)

{

операторы_тела_функции

}

Существует три способа перегрузки:

- оператор-функция определяется как функция, не принадлежащая классу;
- оператор-функция определяется как функция класса;

- оператор-функция определяется как дружественная функция класса.
Особенности перегрузки операций:
- можно перегружать только стандартные операции, например, нельзя перегрузить операцию ‘**’ (возведение в степень в языке Фортран, отсутствует в Си++);
- не допускают перегрузки операции: ‘.’, ‘.*’, ‘?:’, ‘::’, ‘sizeof’, ‘#’, ‘##’;
- при перегрузке сохраняется арность операций (унарная операция остается унарной, а бинарная – бинарной);
- бинарная операция перегружается либо как функция, не принадлежащая классу с двумя параметрами, один обязательно объект (ссылка на объект) класса, или как функция класса с одним параметром, первым операндом операции выступает объект класса, для которого вызывается функция;
- бинарные операции ‘=’, ‘[]’, ‘->’ должны обязательно определяться как компонентные функции класса;
- унарная операция перегружается либо как функция, не принадлежащая классу с одним параметром - объектом (ссылкой на объект) класса, или как функция класса без параметров, операндом операции выступает объект класса, для которого вызывается функция.

2.3. Задачи и порядок выполнения работы

Студент в работе создает класс и необходимы `V1.print()`;

`v2.print()`;

е оператор- функции для перегрузки заданных в своем варианте операций. Особое внимание обратить на способы перегрузки унарных и бинарных операций и параметры оператор- функций для этих операций, когда оператор функция является членом класса и когда не является. Знать те случаи, когда оператор функция должна быть обязательно членом класса и случаи, когда оператор- функция обязательно не принадлежит классу. При защите работы необходимо обосновать выбор способа определения оператор- функции – внутри класса или вне его. При необходимости студент выполняет ручной расчет для проверки работы программы для задачи небольшой размерности. Результаты работы программы, ручного расчета представляются в отчете.

Условие задачи:

Дан класс (например, с именем *Vector*), задающий вектор размерности *n*. Поля класса: указатель на массив, задающий вектор (тип элемента *double*), массив должен

создаваться динамически; число элементов (размерность) вектора (тип *int*). Класс включает: конструктор без параметров, задающий пустой вектор (число элементов равно 0); конструктор, создающий объект вектор на основе обычного одномерного массива размерности *n*; конструктор копирования, конструктор перемещения, деструктор.

Необходимо перегрузить операции и продемонстрировать их работу. Перегрузить операцию `[]` (обращение к элементу вектора по индексу), операцию `=` (копирование вектора или создание копии вектора), операцию `*` (умножение числа на вектора), на выходе вектор такой же размерности, каждый элемент которого равен произведению соответствующего элемента исходного вектора на число.

Пример выполнения работы

Исходный код консольного представлен ниже.

Листинг программы с комментариями:

```
#include <iostream>

using namespace std;
class Vector // Класс- вектор
{
    double *p=nullptr; // Указатель на массив (вектор)
    int n=0; // Размерность вектора (число элементов) массива
public:
    Vector(double *p, int n) // Коструктор на входе массив, задающий вектор
    {
        this->n = n; // Задаем число элементов
        this->p = new double[n]; // Выделяем память
        for (int i = 0; i < n; i++) this->p[i] = p[i]; // Копируем один
        массив в другой
        cout<<"Vector(double *p, int n)"<<endl;
    }
    Vector(int n): n(n) // Конструктор - выделяем память без инициализации
    {
        p=new double[n];
        cout<<"Vector(int n)"<<endl;
    }
    Vector(const Vector & V)
    { // Конструктор копирования
        n = V.n;
        p = new double[n];
        for (int i = 0; i < n; i++)
            p[i] = V.p[i];
        cout<<"Vector(const Vector & V) n="<<n<<endl;
    }
    // Конструктор перемещения (подробная форма)
    /* Vector(Vector&& V) // Параметр - правосторонняя ссылка
       : p(V.p), n(V.n) // Можно инициализировать так
    {
        //p = M.p; // Или инициализировать так
        //n = M.n;
        // Присвойте данным-членам исходного объекта значения по умолчанию.
        V.p = nullptr;
        V.n = 0;
    }*/
    // Конструктор перемещения (краткая форма)
    Vector(Vector&& V) // Параметр - правосторонняя ссылка
```

```

{
    std::swap(p, V.p);
    std::swap(n, V.n);
    cout<<"Vector(Vector&& V)"<<endl;
}
void print() const // Печать вектора (массива), заменить на перегрузку <<
{
    cout<<"n="<<n<<endl;
    for (int i = 0; i < n; i++)
        std::cout << p[i] << " ";
    std::cout << std::endl;
}
Vector() { p = nullptr; n = 0;
    cout<<"Vector()"<<endl;
} // Конструктор без параметров, задает "пустой" объект
double& operator[](int index) // Оператор- функция (перегрузка операции
    // обращения к элементу)
{
    return p[index];
}
Vector & operator =(const Vector& v2) // Оператор- функция копирования
объекта
{
    if (this!=&v2) // Запрет копирования вектора самого в себя
    {
        n = v2.n;
        if (p != nullptr) delete[] p; // Освобождаем память старого
вектора
        p = new double[n]; // Выделяем память для нового вектора
        for (int i = 0; i < n; i++) p[i] = v2.p[i];
    }
    cout<<"Vector & operator =(const Vector& v2)"<<endl;
    return *this; // Возвращаем ссылку на текущий объект
}
Vector & operator =(Vector&& v2) // Оператор- функция перемещения объекта
{
    if (this!=&v2) // Запрет перемещения вектора самого в себя
    {
        /* Подробная форма
        n = v2.n;
        if (p != nullptr) delete[] p; // Освобождаем память старого
вектора
        p = v2.p; // Перемещаем данные
        v2.p=nullptr; // Перемещаемый вектор пуст
        */
        // Краткая форма
        std::swap(p, v2.p);
        std::swap(n, v2.n);
    }
    cout<<"Vector & operator =(Vector&& v2)"<<endl;
    return *this; // Возвращаем ссылку на текущий объект
}
~Vector() // Деструктор
{
    cout<<"~Vector() n="<<n <<endl;
    if (p!=nullptr) delete[] p; // Освобождаем память
}
friend Vector operator *(double x, Vector& v2); // Дружественная функция,
friend Vector operator +(const Vector& v1, const Vector& v2);
// определенная вне класса
};
// Умножение числа на вектор (первый операнд не объект класса,
// функция обязательно определяется вне класса)
Vector operator *(double x, Vector& v2) // Оператор- функция вне класса

```

```

{
    Vector V(v2.n); // Создаем новый объекта заданной размерности
    for (int i = 0; i < v2.n; i++) V.p[i] = x * v2.p[i]; // Заполняем массив
    return V; // Возвращаем объект
}

Vector operator +(const Vector& v1, const Vector& v2) // Оператор- функция
вне класса
{
    Vector V(v1.n+v2.n); // Создаем новый объект заданного размера
    for (int i = 0; i < v1.n; i++) V.p[i] = v1.p[i]; // Заполняем массив
    for (int i = 0; i < v2.n; i++) V.p[i+v1.n] = v2.p[i]; // Заполняем массив
    return V; // Возвращаем объект
}

int main()
{
    double m1[] = { 1, 2, 3, 4.5, 7 };
    Vector V1(m1, 5); // Создаем объект
    Vector V2(m1, 5); // Создаем объект
    Vector V3=V1+V2;
    V3.print();
    return 0;
}

```

2.4. Форма отчета по лабораторной работе

Отчет должен содержать: титульный лист, цель работы, условие задачи, текст программы с комментариями, при необходимости ручной расчет контрольного примера и результаты решения контрольного примера программой для проверки правильности работы алгоритма, выводы по работе.

2.5. Вопросы для самоконтроля

1. Способы перегрузки операций в Си++.
2. Перегрузка бинарной операции, когда первый операнд не является объектом класса.
3. Перегрузка операции с помощью дружественной функции класса.
4. Операции, для перегрузки которых оператор- функция обязательно должна принадлежать классу.

Варианты заданий для лабораторной работы № 2

Дан класс (например, с именем *Vector*), задающий вектор размерности *n*. Поля класса: указатель на массив, задающий вектор (тип элемента *int* или *double* в зависимости от варианта), массив должен создаваться динамически, число элементов (размерность) вектора (тип *int*). Класс включает: конструктор без параметров, задающий пустой вектор (число элементов равно 0), конструктор, создающий объект вектор на основе обычного одномерного массива размерности *n*, конструктор копирования, конструктор перемещения, деструктор.

Необходимо перегрузить операции и продемонстрировать их работу. Перегрузить операцию `[]` (обращение к элементу вектора по индексу), операцию `=` (присваивание с копированием), операцию `=` (присваивание с перемещением), а также операцию вставки (`<<`) объекта в поток `cout` или в файл (объект класса `ostream`) и операцию извлечения (`>>`) объекта из потока `cin` или из файла (объект класса `istream`). *Также продемонстрировать разницу между конструктором копирования и конструктором перемещения и между операциями присваивания с копированием и перемещением. Исходные коды класса разместить в двух файлах: в заголовочном файле класса и файле реализации класса.*

При выполнении работы все входные данные читаются из текстового файла `input.txt` (создать этот файл любым текстовым редактором), результаты выводятся в файл `output.txt`. В отчете представить содержимое этих файлов.

Индивидуальные операции для перегрузки каждым студентом представлены в таблице 1.

Таблица 1 – Варианты заданий

Описание операции перегруженной операции	Тип элемента вектора (массива)	Типы операндов и результата для перегруженной операции			№ варианта
		Первый операнд	Второй операнд	Результат	
+ сложение векторов одинаковой размерности, на выходе вектор такой же размерности элемент которого равен сумме соответствующих элементов двух векторов	double	Vector	Vector	Vector	1
		Vector	double *	Vector	2
		double *	Vector	Vector	3
+ сложение векторов, на выходе вектор, длина которого сумме длин векторов вначале идут элементы первого вектора, затем второго, если один из векторов задан обычным массивом, то считать, что его длина равна длине вектора, заданным объектом класса	double	Vector	Vector	Vector	4
		Vector	double *	Vector	5
		double *	Vector	Vector	6
Постфиксная форма – (декремент), при этом каждый элемент исходного вектора уменьшается на 1, результат операции «старый» (неизменный) вектор.	int	Vector	_____	Vector	7
* умножение вектора на число, на выходе вектор такой же размерности, каждый элемент которого равен произведению соответствующего элемента исходного вектора на число	double	Vector	double	Vector	8
		double	Vector	Vector	9
Префиксная форма ++ (инкремент), при этом каждый элемент исходного вектора увеличивается на 1, результат операции - измененный вектор.	int	Vector	_____	Vector	10
* скалярное произведение векторов (одинаковой размерности), на выходе значение этого произведения	double	Vector	Vector	double	11
		Vector	double *	double	12
		double *	Vector	double	13
Постфиксная форма ++ (инкремент), при этом каждый элемент исходного вектора увеличивается на 1, результат операции «старый» (неизменный) вектор.	int	Vector	_____	Vector	14
^ побитовая операция исключаящая ИЛИ с двумя векторами одинаковой размерности, на выходе вектор такой же размерности элемент, которого равен битовой операции ^ соответствующих элементов двух векторов	int	Vector	Vector	Vector	15
		Vector	int *	Vector	16
		int *	Vector	Vector	17
Префиксная форма -- (декремент),, при этом каждый элемент исходного вектора уменьшается на 1, результат операции измененный вектор.	int	Vector	_____	Vector	18
^ логическая операция (исключающая ИЛИ) с	bool	Vector	Vector	Vector	19

двумя векторами одинаковой размерности, на выходе вектор такой же размерности элемент, которого равен логическая операции $\wedge$ соответствующих элементов двух векторов		Vector	bool *	Vector	20
		bool *	Vector	Vector	21
< сравнение двух векторов (массивов) сравнение проводится последовательно по элементам. Результат истинно, если элемент первого операнда меньше элемента второго элемента, если элемент первого операнда больше элемента второго элемента, результат ложь, в случае равенства переход к следующему паре элементов. Если в каком либо операнде элементы закончились, то результат ложь. Если один из векторов задан обычным массивом, то считать, что его длина равна длине вектора, заданным объектом класса.	int	Vector	Vector	bool	22
		Vector	int *	bool	23
		int *	Vector	bool	24